BLOCKCHAIN Experiment No: 1

Manas Patil
D20A/24

Aim: Cryptography in Blockchain, Merkle root Tree Hash

Tasks Performed:

1. **Hash Generation using SHA-256:**
   - Developed a Python program to compute a SHA-256 hash for any given input string using the hashlib library.
2. **Target Hash Generation with Nonce:**
   - Created a program to generate a hash code by concatenating a user input string and a nonce value to simulate the mining process.
3. **Proof-of-Work Puzzle Solving:**
   - Implemented a program to find the nonce that, when combined with a given input string, produces a hash starting with a specified number of leading zeros.
4. **Merkle Tree Construction:**
   - Built a Merkle Tree from a list of transactions by recursively hashing pairs of transaction hashes, doubling up last nodes if needed, and generated the Merkle Root hash for blockchain transaction integrity.

Program and Output

Program #1: Python program that uses the hashlib library to create the hash of a given string:

```
import hashlib                                                    #error
statement

def create_hash(string):
    # Create a hash object using SHA-256 algorithm
    hash_object = hashlib.sha256()
    # Convert the string to bytes and update the hash object
    hash_object.update(string.encode('utf-8'))
    # Get the hexadecimal representation of the hash
    hash_string = hash_object.hexdigest()                         #error
statement
    # Return the hash string
    return hash_string

# Example usage
input_string = input("Enter a string: ")
hash_result = create_hash(input_string)
print("Hash:", hash_result)                                        #error
statement
```

Output :

```
Enter a string: manas
Hash: 612f14433c2d02e3d4360665272589a6ca9c593b7527ec2d9543376d812f2008
```

Program #2: Program to generate required target hash with input string and nonce

```
import hashlib

# Get user input
input_string = input("Enter a string: ")
nonce = input("Enter the nonce: ")
```

```
# Concatenate the string and nonce
hash_string = input_string + nonce                                    #
Error

# Calculate the hash using SHA-256
hash_object = hashlib.sha256(hash_string.encode('utf-8'))
hash_code = hash_object.hexdigest()

# Print the hash code
print("Hash Code:", hash_code)                                          #
Error
```

Output :

```
Enter a string: manas
Enter the nonce: 1195820
Hash Code:
00000972f021bc657d44277d090bdcfa8445b369f9a719306ffef70db2fdb3ff
```

Program #3: Python code for Solving Puzzle for leading zeros with expected nonce and given string

```
import hashlib

def find_nonce(input_string, num_zeros):
    nonce = 0
    hash_prefix = '0' * num_zeros

    while True:
        # Concatenate the string and nonce
        hash_string = input_string + str(nonce)
#Error
        # Calculate the hash using SHA-256
        hash_object = hashlib.sha256(hash_string.encode('utf-8'))
```

```
        hash_code = hash_object.hexdigest()

        # Check if the hash code has the required number of leading zeros
        if hash_code.startswith(hash_prefix):
#Error
            print("Hash:",hash_code )
            return nonce

        nonce += 1

# Get user input
input_string = "manas"
num_zeros = 5

# Find the expected nonce
expected_nonce = find_nonce(input_string, num_zeros)

# Print the expected nonce
print("Input String:", input_string)
print("Leading Zeros:", num_zeros)
print("Expected Nonce:", expected_nonce)
```

Output :

```
Hash: 00000972f021bc657d44277d090bdcfa8445b369f9a719306ffef70db2fdb3ff
Input String: manas
Leading Zeros: 5
Expected Nonce: 1195820
```

Program 4: Generating Merkle Tree for given set of Transactions

```
import hashlib


# Function to calculate SHA-256 hash
def calculate_hash(data):
    return hashlib.sha256(data.encode('utf-8')).hexdigest()


# Function to build Merkle Tree
```

```python
def build_merkle_tree(transactions):

    if len(transactions) == 0:
        return None

    # Step 1: Hash each transaction
    transactions = [calculate_hash(tx) for tx in transactions]

    print("\nTransaction Hashes:")
    for i, tx_hash in enumerate(transactions):
        print(f"Transaction {i + 1}: {tx_hash}")

    # Step 2: Continue until only one hash remains
    while len(transactions) > 1:

        # If number of hashes is odd, duplicate the last hash
        if len(transactions) % 2 != 0:
            transactions.append(transactions[-1])

        new_transactions = []

        # Combine hashes in pairs
        for i in range(0, len(transactions), 2):

            combined = transactions[i] + transactions[i + 1]

            hash_combined = calculate_hash(combined)

            new_transactions.append(hash_combined)

        transactions = new_transactions

        # Print current level
        print("\nNext Level:")
        for hash_value in transactions:
            print(hash_value)

    # Final hash = Merkle Root
    return transactions[0]
```

```
# ------------------------------
# Example Usage
# ------------------------------

transactions = [
    "Manas -> Chinmay: $400",
    "Bob -> Dave: $500",
    "Dave -> Eve: $100",
    "Eve -> Alice: $300",
    "Roo -> Bob: $50"
]


# Build Merkle Tree
merkle_root = build_merkle_tree(transactions)


# Display Merkle Root
print("\n==============================")
print("Merkle Root:")
print(merkle_root)
print("==============================")
```

Output :

```
Transaction Hashes:
Transaction 1:
f16d247a78d76de30d247f5ec95f36cf31301eb54242a476a9a617f9fd6f5d65
Transaction 2:
fc8102cd3cedc9fdae34f71ad239a15fc6c641dce92d314aa6327ba0e00fbc29
Transaction 3:
f624e157490da71806d3e9d0911c3f5fa53f69edf1b53566fea0ef82a2a6ce3c
Transaction 4:
6f881bd56fd9d5cb5ec7ce2e4fb5bd5247d387942280990ee1d260cee02bc1f7
Transaction 5:
3cb00b6c868029bce21986e2b845a2d47788fbb029bd12b7f6ea9e253c69e6f2

Next Level:
```

```
98f7de93f1aab9961a8deb75a293c8d6245af6591b4833a105f62c4d32af5e8c
c5b0544ec31f6faf0e5f5f658287245a6a6231ea359cbd4ddbde9b8c6fab072e
9295aaa068e5269a09c74898234d0a2f28460d47b1fa5df08e439de44222651e

Next Level:
cf66ecbdfdd43e9cfc8c8d8b4c7417403e66cb70c4fa7e6ed9cab0d39e5c356a
192f5368654b32c94fedaa68e3f106c425cf3ab1ec5b8465fcc33acd30140f65

Next Level:
76f05a112ac75e1ee1b0c2d1e91a17b11ea8ccccdd48cf06e2d87c85e747fd93

==============================
Merkle Root:
76f05a112ac75e1ee1b0c2d1e91a17b11ea8ccccdd48cf06e2d87c85e747fd93
==============================
```

Conclusion:

This study demonstrates fundamental cryptographic concepts and their applications within blockchain technology:

- **SHA-256 Hashing** provides a secure and unique fingerprint for any input, ensuring data integrity.
- **Nonce and Target Hashing** allow the simulation of proof-of-work consensus, emphasizing the difficulty of mining blocks by meeting hash conditions (leading zeros).
- **Proof-of-Work Puzzle Solving** shows how mining requires computational effort to find a nonce making the hash satisfy network difficulty.
- **Merkle Tree Construction** efficiently summarizes and validates large numbers of transactions with a single Merkle root hash, which is essential in blockchain for fast verification and ensuring the integrity of data blocks.

Together, these implementations capture the essence of how cryptography underpins blockchain's security, immutability, and efficiency, especially through hash functions and structures like Merkle Trees that enable trustless and decentralized transaction management.